Daïzer Bercy

Rapport Projet système et réseau

Comment exécuter le programme.

Faire un make pour créer les exécutables.

```
→  Wikicomptines git:(main) make
mkdir -p build
gcc -Iinclude -g -Wall -c src/wcp_clt.c -o build/wcp_clt.o
gcc -Iinclude -g -Wall -c src/comptine_utils.c -o build/comptine_utils.o
gcc  build/wcp_clt.o build/comptine_utils.o -o wcp_clt -pthread
gcc -Iinclude -g -Wall -c src/wcp_srv.c -o build/wcp_srv.o
gcc  build/wcp_srv.o build/comptine_utils.o -o wcp_srv -pthread
```

Ensuite, il faut ouvrir deux terminaux.
Dans le premier terminal pour le serveur, il faut écrire :

```
→  Wikicomptines git:(main) ./wcp_srv comptines
Serveur en attente de connexions...
```

Dans le deuxième terminal pour le client, il faut écrire :

```
→  Wikicomptines git:(main) ./wcp_clt 127.0.0.1
Connexion établie avec le serveur 127.0.0.1:4321
     0 Mon petit lapin
     1 Dans sa maison, un grand cerf
     2 Petit escargot
     3 La famille tortue
     4 Les petits poissons dans l'eau
     5 G comme Gaston
Quelle comptine voulez-vous ?(Entrer un entier entre 0 et 5) :
```

Il faut ensuite choisir la comptine et elle s'affichera.

```
Quelle comptine voulez-vous ?(Entrer un entier entre 0 et 5) : 3
La famille tortue

Jamais on a vu
Jamais on ne verra
La famille tortue
Courir après les rats

Le papa tortue
La maman tortue
Les enfants tortues
Iront toujours
Au pas
```

Au niveau du serveur, des messages s'afficheront en fonction des actions qu'il aura effectuées.

```
→  Wikicomptines git:(main) ./wcp_srv comptines
Serveur en attente de connexions...
Connexion acceptée
liste envoyée
comptine envoyée
```

À noter que le serveur ne s'arrête pas tous seul, il faudra donc faire un « ctrl + c» pour l'arrêter.

Description des fonctionnalités mise en œuvre avec quelques éléments permettant de comprendre rapidement votre implémentation.

Tout d'abord, il n'a pas eu d'ajout au protocole WCP dans ce projet, le strict minimum, c'est-à-dire la section 2, 3 et 4, on été fait.

Pour le fichier comptine_utils.c:

- Pour la fonction :

```c
int read_until_nl(int fd, char *buf)
{
   int i=0;

   while((read(fd,buf + i,1) == 1) && (buf[i]!='\n')){
      i++;
   }

   return i;
}
```

La fonction read est utilisé dans une condition de boucle while pour lire les caractères 1 à 1 jusqu'à rencontré un saut de ligne.

- Pour la fonction:

```c
int est_nom_fichier_comptine(char *nom_fich)
{
   char cpt[4];
   int size = strlen(nom_fich),j=0,i;

   for(i=size-4;i<size+1;i++){
      cpt[j++] = nom_fich[i];
   }

   if(strcmp(cpt,".cpt") == 0){
      return 1;
   }
   return 0;
```

```
}
```

Ici, j'ai utilisé un tableau nommé cpt dans lequel, je vais stocker les quatre derniers caractère du paramètre d'entrée que j'ai réussi à avoir avec la fonction strlen. Puis je vais comparer grâce à la fonction strcmp les 4 dernier caractère du paramètre d'entrée avec la chaîne «.cpt».

- Pour la fonction:

```
struct comptine *init_cpt_depuis_fichier(const char *dir_name, const char
*base_name)
//chemin ~/dir_name/file
    char chemin[256];
    strcpy(chemin,dir_name);
    strcat(chemin,"/");
    strcat(chemin,base_name);
    // open
    int fd;
    if((fd=open(chemin,O_RDONLY))==-1){
        perror("Open");
        return NULL;
    }
    struct comptine *cpt=malloc(sizeof(struct comptine *));
    if (cpt == NULL) {
        perror("malloc");
        close(fd);
        return NULL;
    }
```

Ici, un tableau chemin a été initialisé pour avoir le chemin entier du fichier, c'est-à-dire avec son répertoire. Pour ensuite l'ouvrir avec un open.
Un structure comptine est ensuite initialisé puis alloué puis ensuite vérifié.

```
//nom_fichier
cpt->nom_fichier = malloc(sizeof(char *) *(strlen(base_name) + 1));
    if (cpt->nom_fichier == NULL) {
        perror("malloc");
        close(fd);
        free(cpt);
        return NULL;
    }
    strcpy(cpt->nom_fichier,base_name);
```

Ensuite le nom_fichier dans la structure de la comptine est alloué pour être initialisé avec le paramètre d'entrée base_name qui est le nom du

fichier. Que je vais copier dedans avec la fonction strcpy.

```
//pour le titre
   char buf[256];
   int n = read_until_nl(fd,buf);
   if(n == -1){
      perror("read_until_nl");
      close(fd);
      return NULL;
   }
   buf[n] = '\0';
   cpt->titre = malloc(sizeof(char *) * (n+2));
    if (cpt->titre == NULL) {
      perror("malloc");
      close(fd);
      free(cpt->nom_fichier);
      free(cpt);
      return NULL;
   }
   strcpy(cpt->titre,buf);

   close(fd);
   return cpt;
}
```

Enfin, pour le titre, j'ai créé un autre tableau buf, on utilise read_utils_nl pour avoir la taille et lire le titre dans le buf. Puis on initialise le titre à la bonne taille grâce à l'allocation dynamique. Pour ensuite copier le buf dans le titre de la structure comptine.

- Pour la fonction :

```
struct catalogue *creer_catalogue(const char *dir_name)
//open
   DIR *dir = opendir(dir_name);
   if (dir == NULL) {
      perror("opendir");
      return NULL;
   }
   //taille cat->nb
   struct dirent* ent;
   int i = 0;
   while ((ent = readdir(dir)))
      if (est_nom_fichier_comptine(ent->d_name))
         i++;
```

```
    // retourne au debut du repertoire
    rewinddir(dir);
```

Ici, la fonction opendir est utilisé pour ouvrir le répertoire ensuite, on fait une boucle while qui servira plus tard à initialiser le nombre de catalogues. La boucle while regarde le nombre de comptines qu'il y a dans le répertoire.
On utilise ensuite la fonction rewinddir pour revenir au début du répertoire.

```
//initialisation avec les bonnes tailles
    struct catalogue* cat =malloc(sizeof(struct catalogue * ));
    if(cat == NULL){
        perror("malloc");
        closedir(dir);
        return NULL;
    }

    cat->nb = i;
    cat->tab = malloc(sizeof(struct comptine *) * i);
    if(cat->tab == NULL){
        perror("malloc");
        closedir(dir);
        free(cat);
        return NULL;
    }
```

Initialisation du catalogue et de son tableau avec les bonnes tailles, ainsi que leur vérification en cas d'erreur lors de l'allocation dynamique.

```
// on remplit le tableau qui est de struct comptine titre et nom du fichier
    i = 0;
    while ((ent = readdir(dir))) {
        if (est_nom_fichier_comptine(ent->d_name)) {
            cat->tab[i] = init_cpt_depuis_fichier(dir_name, ent->d_name);
            // cas d'erreur
            if(cat->tab[i] == NULL){
                perror("NULL");
                liberer_catalogue(cat);
                closedir(dir);
                return NULL;
            }
            i++;
        }
    }
        closedir(dir);
```

```
    return cat;
}
```

Puis enfin une fois être revenu au début du répertoire grâce à rewinddir, on relit le répertoire pour initialiser le tableau du catalogue qui est lui-même une structure de type comptine. On utilise la fonction init_cpt_depuis_fichier qui initialise les comptines et qui a été implémenté un peu avant. On fait ensuite un cas pour géré les erreurs et on ferme le répertoire grâce à l'appel de la fonction closedir.

Pour le fichier client wcp_clt.c :

- Pour la fonction :

```
int creer_connecter_sock(char *addr_ipv4, uint16_t port)
// Création du socket partie client
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }
```

On commence par initialiser le socket avec la fonction socket qui prend en argument AF_INET pour dire qu'on utilisera une adresse IPv4, et SOCK_STREAM pour préciser que le protocole sera TCP. Suivi de 0 pour le protocole, car on souhaite crée un socket de utilisant le protocole TCP avec une adresse IPv4.

```
// Création de la structure sockaddr_in pour la liaison
    struct sockaddr_in sa;
    sa.sin_family = AF_INET;
    sa.sin_port = htons(port);
    if(inet_pton(AF_INET, addr_ipv4, &sa.sin_addr)!=1){
        fprintf(stderr, "adresse ipv4 non valable\n");
        exit(1);
    }
```

Ici, on initialise la structure sockaddr_in car on est en IPv4 la famille d'adresse sera AF_INET (IPv4), le port sera celui du paramètre de la fonction, on utilise htons, pour convertit le port de l'ordre d'octets de l'hôte à l'ordre d'octets du réseau, pour garantir que le port soit lu correctement.
On utilise la fonction inet_pton pour initialiser sin_addr. Cette fonction selon le manuel va convertir la chaîne de caractère add_ipv4 qui en paramètre d'entrée en adresse réseau IPv4, ici car on a mis AF_INET en premier argument puis un fois la conversion fini, il va copier dans sin_addr.

```
 socklen_t sl = sizeof(sa);

   // connection côté client
   if(connect(sock, (struct sockaddr *) &sa, sl) < 0){
      perror("connect");
      exit(EXIT_FAILURE);
   }
   return sock;
}
```

Enfin, une fois le socket créé et initialisé, on crée la connexion en utilisant la fonction connect qui prend en premier argument, le descripteur du fichier socket, le deuxième argument est la structure à laquelle le descripteur sera connecter. Avec la taille de la structure en troisième argument .

- Pour la fonction :

```
uint16_t recevoir_liste_comptines(int fd)
{
   char buf[256];
   uint16_t i=0;
   int n=0;
   while((n=read_until_nl(fd,buf)) !=0){
      buf[n+1] = '\0';
      printf("%s",buf);
      i++;
   }

   return i;
}
```

Ici, initialisation d'un tableau buf qui servira à afficher la comptine avec l'appel de la fonction read_until_nl pour lire dans le buf. Et qui retournera la taille de la liste qui sera incrémentée à chaque lecture de titre.

- Pour la fonction :

```
uint16_t saisir_num_comptine(uint16_t nb_comptines)
{
   uint16_t n;
   do{
      printf("Quelle comptine voulez-vous ?(Entrer un entier entre 0 et
%d) : ",nb_comptines - 1);
      scanf("%hu",&n);
   } while(n>=nb_comptines);
```

```
    return n;
}
```
On utilise ici, un scanf dans un do-while est utilisé tant que le nombre entrer est entre 0 et le nb en paramètre de la fonction moins 1.

- Pour la fonction :

```
void envoyer_num_comptine(int fd, uint16_t nc)
{
    uint16_t n = htons(nc);
    write(fd,&n,sizeof(n));
}
```
On utilise htons ici pour convertir l'entrée nc pour ensuite l'écrire dans le descripteur. Pour envoyer une demande au serveur.

- Pour la fonction :

```
void afficher_comptine(int fd)
{
    char buff[256];
    int n;
    while((n=read(fd,buff,sizeof(buff)))>0){
        for(int i=0; i<n; i++){
            if(buff[i] == '\n'){
                printf("\n");
            }else {
                printf("%c", buff[i]);
            }
        }
    }

    if(n<0){
        perror("Erreur lecture du descripteur de fichier");
    }
}
```
Ici, on se contente de lire les comptines avec un read et un printf du buf dans des boucles.

Pour le fichier wcp_srv.c :

Pour la fonction :

```
int creer_configurer_sock_ecoute(uint16_t port)
int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(2);
```

```
    }
    struct sockaddr_in sa;
    sa.sin_family = AF_INET;
    sa.sin_port = htons(port);
    sa.sin_addr.s_addr=htonl(INADDR_ANY);
```

Ici, comme habituellement, on initialise un socket avec la structure, ici, on est sur de l'IPv4 (AF_INET) et sur un protocole TCP (SOCK_STREAM). Petite différence avec le client, c'est que là .sin_addr reçoit toutes les connexions grâce à INADDR_ANY.

```
if(bind(sock,(struct sockaddr *) &sa, sizeof(sa)) < 0){
        perror("bind");
        exit(3);
    }

    if(listen(sock,128)< 0 ){
        perror("listen");
        exit(2);
    }
    return sock;
}
```

Un bind est ensuite fait pour lier le socket à l'adresse pour ensuite faire un listen qui servira à accepter les nouvelles connexions avec une liste d'attente de 128 ici.

Pour la fonction :

```
void envoyer_liste(int fd, struct catalogue *c)
{
    for(int i=0 ;i<c->nb ;i++){
        dprintf(fd, "%6d %s\n",i,c->tab[i]->titre);
    }
    dprintf(fd,"\n");
}
```

On enverra la liste du catalogue. On utilise ici dprintf pour écrire dans le descripteur de fichier.

Pour la fonction:

```
uint16_t recevoir_num_comptine(int fd)
{
    uint16_t n;
    read(fd,&n,sizeof(uint16_t));
    n = ntohs(n);
    return n;
}
```

On recevra le numéro que le client nous enverra lors du choix de la comptine.

Enfin pour la fonction :

```
oid envoyer_comptine(int fd, const char *dirname, struct catalogue *c,
uint16_t ic)
{   // chemin du fichier pour l'ouvrir
    char chemin[256];
    strcpy(chemin,dirname);
    strcat(chemin,"/");
    strcat(chemin,c->tab[ic]->nom_fichier);
    int fd2;
    if((fd2=open(chemin, O_RDONLY)) == -1){
        perror("open");
        return;
    }

    // écriture de la comptine dans fd
    int n;
    char buf[256];
    while((n=read(fd2,buf,sizeof(buf)))>0){
        write(fd,buf,n);
    }
    write(fd,"\n\n",2);
    close(fd2);
}
```

On sélectionnera le chemin de la bonne comptine choisi par le client. Pour l'ouvrir avec open. Puis on initialisera un tableau buf qui servira à lire la comptine. On utilise le descripteur qui a ouvert la bonne comptine pour qu'ils lisent le contenu dans un buf avec read puis avec ce buf on écrit dans le descripteur en paramètre d'entrée avec un write.

<u>Au niveau de l'ordre du fonctionnement du programme :</u>

- : Serveur
- : Client

- On commence avec le serveur qui attente la connexion d'un hôte (client).

```
sock_srv = creer_configurer_sock_ecoute(PORT_WCP);
```

- Un client se connecte.

```
int sock = creer_connecter_sock(argv[1], PORT_WCP);
```

- Le serveur accepte ou refuse.

```
sock_clt = accept(sock_srv,(struct sockaddr *) &sa_clt, &sl);
```

- S'il accepte, il envoie la liste des comptines.

```
 envoyer_liste(sock_clt,c);
```

- Le client reçoit la liste.

```
uint16_t nb = recevoir_liste_comptines(sock);
```

- Le client (hôte) choisit la comptine et l'envoie au serveur.

```
uint16_t n = saisir_num_comptine(nb);
envoyer_num_comptine(sock,n);
```

- Le serveur reçoit le numéro de comptine.

```
uint16_t recp_num_comp = recevoir_num_comptine(sock_clt);
```

- Le serveur envoie la comptine au client.

```
envoyer_comptine(sock_clt,argv[1],c,recp_num_comp);
```

- Le client affiche.

```
afficher_comptine(sock);
```

Ressource externe au cours utilisé pour le projet :

- https://moodlelms.univ-paris13.fr/pluginfile.php/622293/mod_resource/content/14/td04-corr.pdf
- http://manpagesfr.free.fr/man/man2/socket.2.html
- http://manpagesfr.free.fr/man/man2/accept.2.html
- http://manpagesfr.free.fr/man/man2/connect.2.html
- http://manpagesfr.free.fr/man/man2/listen.2.html
- http://manpagesfr.free.fr/man/man2/bind.2.html
- https://www.codequoi.com/manipuler-un-fichier-a-laide-de-son-descripteur-en-c/
- https://www.codequoi.com/programmation-reseau-via-socket-en-c/

En conclusion, ce projet a été un excellent moyen pour moi de mieux comprendre le principe de socket et son fonctionnement.